

Analisis dan Diskusi Kontrol Cerdas

Minggu ke - 4

Nama : Rani Dwi Setiani
NIM : 224308043
Kelas : TKA-6B
Akun Github : <https://github.com/Mutsee>
Student Lab Assistant : Mas Alfian

✦ Analisis Hasil:

- **Bagaimana performa agen dalam mengontrol environment CartPole?**

Kinerja agen dalam mengontrol lingkungan **CartPole** dapat dievaluasi berdasarkan seberapa baik agen tersebut dapat menjaga tiang vertikal tetap tegak dengan memindahkan kereta (cart) ke kiri atau ke kanan. Dalam hal ini, agen biasanya berfungsi untuk memutuskan tindakan apa yang harus diambil (gerakan kiri atau kanan) berdasarkan posisi dan kecepatan tiang serta kereta. Dalam lingkungan CartPole, agen biasanya menggunakan algoritma pembelajaran penguatan (reinforcement learning) untuk belajar dari interaksinya dengan lingkungan. Berikut adalah beberapa aspek yang mempengaruhi kinerja agen:

1. **Pengamatan Lingkungan:** Agen mendapatkan pengamatan yang terdiri dari empat elemen: posisi kereta, kecepatan kereta, sudut tiang, dan kecepatan sudut tiang. Agen harus menginterpretasikan informasi ini untuk membuat keputusan yang tepat.
2. **Tindakan yang diambil:** Agen dapat mengambil tindakan berupa gerakan kiri atau kanan pada kereta. Dengan tindakan yang benar, agen dapat mempertahankan keseimbangan tiang lebih lama.
3. **Penerapan Algoritma Pembelajaran Penguatan:** Algoritma yang digunakan untuk mengontrol agen dapat bervariasi, misalnya:
 - **Q-Learning** atau **Deep Q-Network (DQN)** untuk menentukan kebijakan yang optimal berdasarkan nilai Q.
 - **Policy Gradient Methods** seperti REINFORCE atau Proximal Policy Optimization (PPO), yang mempelajari kebijakan langsung tanpa memerlukan fungsi nilai terpisah.
4. **Hadiah (Reward):** Lingkungan CartPole memberikan hadiah positif ketika agen berhasil menjaga tiang tegak, dan menghukum agen

dengan pengurangan poin atau akhir episode jika tiang jatuh atau keluar dari batasan yang ditentukan.

5. **Stabilitas dan Waktu Episode:** Kinerja agen dapat diukur berdasarkan waktu yang dibutuhkan untuk menjaga tiang tetap tegak. Agen yang lebih efektif dapat bertahan lebih lama dalam menjaga tiang agar tidak jatuh, misalnya mencapai lebih dari 200 langkah dalam satu episode (pada standar tertentu).

Evaluasi Kinerja:

- **Konsistensi:** Agen yang baik akan mampu mengontrol CartPole secara konsisten, menjaga keseimbangan selama waktu yang lebih lama.
- **Penggunaan Sumber Daya:** Sebuah agen yang lebih efisien dalam pelatihan akan membutuhkan lebih sedikit iterasi untuk mencapai kinerja yang optimal.

Jadi, kinerja agen tergantung pada algoritma yang digunakan, parameter yang dipilih, dan pengalaman agen dalam berinteraksi dengan lingkungan. Sebuah agen yang sudah terlatih dengan baik akan mampu menjaga tiang tetap tegak dalam waktu yang lama, bahkan tanpa banyak perubahan dalam kecepatan atau posisi kereta.

- **Bagaimana perubahan parameter (misal: gamma, epsilon, learning rate) mempengaruhi kinerja agen?**

Perubahan parameter dalam algoritma pembelajaran penguatan (*reinforcement learning*) seperti **gamma (discount factor)**, **epsilon (exploration rate)**, dan **learning rate** memiliki dampak signifikan terhadap kinerja agen, termasuk dalam lingkungan seperti **CartPole**. **Gamma (γ)** mengontrol seberapa besar agen memberi bobot pada hadiah masa depan dibandingkan dengan hadiah yang diterima saat ini. Dengan γ yang lebih tinggi, agen akan lebih fokus pada hasil jangka panjang, tetapi dapat mengarah pada pembelajaran yang lebih lambat, terutama jika penghargaan instan kurang dihargai. Sebaliknya, dengan γ yang lebih rendah, agen lebih fokus pada penghargaan instan, yang bisa mengurangi kemampuan perencanaan jangka panjang. **Epsilon (ϵ)** menentukan seberapa sering agen mengeksplorasi tindakan acak dibandingkan dengan mengeksploitasi kebijakan yang telah dipelajari. Dengan nilai ϵ yang tinggi, agen lebih banyak mengeksplorasi, membantu menemukan kebijakan optimal, tetapi bisa memperlambat konvergensi. Di sisi lain, nilai ϵ yang rendah mendorong agen untuk lebih sering mengeksploitasi kebijakan terbaik yang telah ditemukan, mempercepat konvergensi namun berisiko terjebak dalam kebijakan suboptimal. Penggunaan **epsilon decay**, yang menurunkan ϵ secara bertahap selama pelatihan, memungkinkan agen untuk mulai dengan

eksplorasi yang lebih besar dan beralih ke eksploitasi seiring waktu. Sedangkan **learning rate (α)** mengontrol seberapa cepat agen memperbarui perkiraan nilai Q atau kebijakan berdasarkan pengalaman baru. Nilai α yang tinggi dapat mempercepat pembelajaran, namun bisa menyebabkan ketidakstabilan, sedangkan nilai α yang rendah memperlambat pembelajaran tetapi menghasilkan proses yang lebih stabil. Kombinasi yang tepat dari ketiga parameter ini sangat penting untuk mencapai kinerja optimal, dengan eksperimen dan penyesuaian yang diperlukan untuk tugas tertentu. Sebagai contoh, dalam lingkungan CartPole, agen yang terlalu cepat mengeksploitasi atau terlalu fokus pada penghargaan jangka panjang tanpa cukup mengeksplorasi bisa mengalami kesulitan dalam mencapai kebijakan optimal. Oleh karena itu, penyesuaian parameter yang cermat dapat membantu agen menyesuaikan strategi belajar dan meningkatkan kinerjanya.

- **Apa tantangan yang muncul selama pelatihan agen RL?**

Pelatihan agen **Reinforcement Learning (RL)** dapat menghadapi sejumlah tantangan yang dapat mempengaruhi keberhasilan dan efisiensi agen dalam mempelajari kebijakan optimal. Beberapa tantangan utama yang sering muncul selama pelatihan agen RL meliputi:

1. Exploration vs. Exploitation Dilemma

Salah satu tantangan paling mendasar dalam RL adalah **dilema eksplorasi versus eksploitasi**. Agen harus menyeimbangkan antara mengeksplorasi lingkungan (mencoba tindakan yang belum dipelajari) untuk menemukan kebijakan optimal dan mengeksploitasi pengetahuan yang sudah ada (memilih tindakan yang diketahui memberikan hasil terbaik). Terlalu banyak eksplorasi dapat mengarah pada pembelajaran yang lambat, sementara terlalu banyak eksploitasi dapat menyebabkan agen terjebak dalam kebijakan suboptimal.

2. Sparsity of Rewards (Kelangkaan Hadiah)

Dalam banyak tugas RL, agen hanya menerima hadiah setelah sejumlah langkah atau episode tertentu. Hal ini bisa membuat agen kesulitan dalam mengaitkan tindakan spesifik dengan hasil yang diinginkan, terutama dalam masalah yang lebih kompleks di mana hadiah hanya diberikan setelah serangkaian tindakan atau setelah mencapai tujuan tertentu. **Sparsity of rewards** dapat membuat pelatihan sangat lambat dan tidak stabil, karena agen harus belajar dari sedikit informasi tentang apakah tindakannya benar atau salah.

3. Credit Assignment Problem

Tantangan ini terjadi ketika agen harus menentukan tindakan mana yang menghasilkan hasil tertentu, terutama jika penghargaan diberikan

setelah serangkaian langkah. Misalnya, jika agen menyelesaikan suatu tugas setelah 100 langkah, sulit untuk menentukan tindakan mana dari 100 langkah tersebut yang benar-benar berkontribusi terhadap keberhasilan. Ini mengarah pada masalah **credit assignment**, di mana agen kesulitan untuk memberi nilai yang tepat pada tindakan yang telah diambil sebelumnya, terutama dalam masalah dengan banyak langkah atau episode panjang.

4. High Variance in Learning

Dalam pembelajaran penguatan, **fluktuasi besar dalam hasil** sering terjadi, terutama ketika agen mempelajari kebijakan melalui pengalaman langsung. Hal ini bisa menyebabkan ketidakstabilan dalam proses pelatihan. Misalnya, perubahan dalam tindakan yang dipilih oleh agen bisa menyebabkan variansi tinggi dalam hasil, yang membuat pelatihan lebih sulit dan membutuhkan waktu lebih lama untuk mengonvergensi ke kebijakan yang baik.

5. Overfitting

Agen RL juga dapat mengalami **overfitting**, terutama ketika model yang digunakan terlalu kompleks dan dilatih terlalu lama pada lingkungan yang terbatas. Hal ini bisa menyebabkan agen terlalu bergantung pada pola yang ada dalam data pelatihan dan tidak dapat menggeneralisasi dengan baik ke situasi atau keadaan baru dalam lingkungan yang lebih luas atau berbeda.

6. Kompleksitas Lingkungan

Dalam beberapa tugas RL, **kompleksitas lingkungan** bisa menjadi masalah besar. Jika lingkungan sangat besar atau memiliki banyak dimensi (seperti dalam permainan video 3D atau kontrol robot), proses pelatihan menjadi jauh lebih rumit. Agen harus mempelajari kebijakan dalam ruang aksi dan keadaan yang sangat besar, yang dapat meningkatkan waktu pelatihan secara signifikan. Selain itu, interaksi yang kompleks antara elemen-elemen dalam lingkungan membuatnya lebih sulit bagi agen untuk menemukan pola yang berguna.

7. Partial Observability (Pengamatan Parsial)

Beberapa tugas RL melibatkan **pengamatan parsial** di mana agen hanya dapat melihat sebagian kecil informasi tentang keadaan lingkungan. Ini menyebabkan agen harus membuat keputusan dengan informasi yang tidak lengkap, yang meningkatkan kesulitan dalam mempelajari kebijakan yang baik. Dalam hal ini, agen perlu mengembangkan strategi untuk menangani ketidakpastian dan memanfaatkan informasi yang terbatas sebaik mungkin.

8. Stabilitas dan Kecepatan Konvergensi

Pembelajaran dalam RL bisa sangat **tidak stabil**, terutama ketika menggunakan algoritma seperti Q-learning atau pembelajaran berbasis jaringan syaraf dalam kasus Deep Reinforcement Learning. Jaringan neural yang besar dan kompleks rentan terhadap masalah **overfitting** dan **vanishing/exploding gradients**, yang dapat memperlambat proses konvergensi dan mengarah pada pembelajaran yang tidak stabil. Pemilihan parameter yang tepat, seperti learning rate dan discount factor, sangat penting untuk mencapai konvergensi yang stabil.

9. Keterbatasan Komputasi

Tugas-tugas RL, terutama dalam Deep RL, sering kali memerlukan **sumber daya komputasi yang besar**. Pelatihan agen pada lingkungan yang kompleks atau dengan model yang sangat besar dapat memerlukan waktu yang lama dan biaya komputasi yang tinggi. Keterbatasan ini membuat eksperimen dengan parameter yang berbeda menjadi lebih sulit dan mahal.

10. Tuning Hyperparameters

Pemilihan dan penyesuaian **hyperparameter** dalam algoritma RL sangat krusial. Parameter seperti learning rate, discount factor (γ), dan epsilon (ϵ) dapat memiliki dampak besar terhadap kinerja agen. Menemukan kombinasi yang optimal sering kali melibatkan pencarian eksperimen yang ekstensif dan membutuhkan banyak waktu dan upaya, terutama ketika lingkungan yang digunakan sangat besar dan kompleks.

11. Non-Stationary Environments

Beberapa lingkungan RL bersifat **non-stationary**, yang berarti bahwa dinamis lingkungan berubah seiring waktu. Dalam kasus ini, kebijakan yang dipelajari oleh agen mungkin menjadi kurang relevan seiring perubahan kondisi, yang mengharuskan agen untuk terus-menerus menyesuaikan dirinya dengan keadaan yang berubah. Ini bisa menambah tantangan dalam mempertahankan kinerja yang baik di seluruh proses pelatihan.

12. Sample Efficiency

Agan RL umumnya membutuhkan banyak **interaksi dengan lingkungan** untuk mempelajari kebijakan yang efektif. Dalam beberapa kasus, ini dapat sangat tidak efisien, baik dari sisi waktu maupun sumber daya. Penggunaan metode seperti **model-based RL**, **imitation learning**, atau **prioritized experience replay** dapat membantu meningkatkan efisiensi sampel, tetapi ini juga memperkenalkan tantangan baru dalam hal implementasi dan stabilitas.

Kesimpulan

Pelatihan agen RL menghadapi banyak tantangan, mulai dari dilematis antara eksplorasi dan eksploitasi, hingga masalah yang lebih teknis seperti kestabilan pembelajaran dan keterbatasan komputasi. Mengatasi tantangan-tantangan ini sering membutuhkan teknik-teknik lanjutan seperti penurunan epsilon secara bertahap, pembelajaran berbasis model, atau penggunaan algoritma pembelajaran yang lebih efisien seperti **Proximal Policy Optimization (PPO)** atau **Trust Region Policy Optimization (TRPO)**.

Diskusi:

- **Apa perbedaan utama antara Reinforcement Learning dan metode supervised learning dalam sistem kendali?**

Perbedaan utama antara **Reinforcement Learning (RL)** dan **Supervised Learning** dalam konteks sistem kendali terletak pada cara kedua metode ini belajar, jenis umpan balik yang diterima, dan cara mereka digunakan untuk mengontrol sistem. Dalam **Reinforcement Learning (RL)**, agen belajar melalui interaksi langsung dengan lingkungan, di mana umpan balik yang diterima berupa penghargaan atau hukuman setelah agen mengambil tindakan, dan umpan balik ini sering kali tidak langsung serta tidak terlabeli. Agen harus belajar mengoptimalkan kebijakan untuk memaksimalkan reward jangka panjang berdasarkan pengalaman. Sebaliknya, dalam **Supervised Learning**, agen atau model belajar dari data yang sudah terlabeli, di mana setiap input sudah memiliki output yang benar. Proses pembelajaran berfokus pada pencocokan antara input dan output yang sudah diketahui, dan model tidak perlu berinteraksi dengan lingkungan secara langsung. Dalam sistem kendali, RL digunakan untuk belajar mengendalikan sistem yang tidak sepenuhnya diketahui atau sulit dimodelkan, terutama dalam lingkungan yang dinamis dan berubah, sementara Supervised Learning lebih efektif dalam mengontrol sistem yang lebih stabil, di mana hubungan antara input dan output sudah dipahami atau diperkirakan. RL sangat bergantung pada eksplorasi dan penyesuaian terhadap perubahan dalam lingkungan, sementara Supervised Learning membutuhkan data terlabeli yang sudah ada dan tidak dapat beradaptasi dengan perubahan tanpa pelatihan ulang. Dengan demikian, RL cocok untuk sistem kendali yang membutuhkan adaptasi dinamis, sedangkan Supervised Learning lebih tepat untuk sistem yang dapat diprediksi dan terstruktur.

- **Bagaimana strategi eksplorasi (exploration) dan eksploitasi (exploitation) dapat dioptimalkan pada agen RL?**

Strategi eksplorasi (exploration) dan eksploitasi (exploitation) adalah dua komponen utama dalam **Reinforcement Learning (RL)** yang perlu dioptimalkan agar agen dapat belajar dengan efektif. Eksplorasi mengacu pada upaya agen untuk mencoba tindakan baru yang belum dipelajari, sementara eksploitasi adalah upaya agen untuk memilih tindakan yang

sudah diketahui memberikan hasil terbaik. Salah satu cara yang umum digunakan untuk menyeimbangkan keduanya adalah dengan menggunakan strategi **epsilon-greedy**, di mana agen memilih tindakan acak dengan probabilitas ϵ (eksplorasi) dan memilih tindakan terbaik dengan probabilitas $1 - \epsilon$ (eksploitasi). Nilai ϵ biasanya dimulai dengan nilai tinggi dan kemudian dikurangi seiring waktu, dalam proses yang disebut **epsilon decay**, untuk mendorong lebih banyak eksploitasi setelah agen mulai memperoleh pengetahuan. Metode lain yang lebih canggih adalah **Upper Confidence Bound (UCB)**, yang memilih tindakan berdasarkan nilai ekspektasi dan ketidakpastian tentang reward yang diperkirakan, sehingga mendorong eksplorasi pada tindakan yang belum dipelajari secara memadai. **Thompson Sampling** adalah pendekatan probabilistik di mana agen memilih tindakan berdasarkan distribusi probabilistik reward yang diperkirakan, mendorong eksplorasi lebih banyak di awal dan eksploitasi saat distribusi reward semakin jelas. **Boltzmann Exploration** atau **Softmax** menggunakan suhu untuk mengatur probabilitas eksplorasi dan eksploitasi, dengan suhu tinggi mendorong eksplorasi dan suhu rendah mendorong eksploitasi. Pendekatan **intrinsic motivation**, seperti eksplorasi berbasis rasa ingin tahu, memberi agen penghargaan internal untuk mengeksplorasi keadaan baru, mendorong eksplorasi yang lebih banyak di lingkungan dengan penghargaan yang langka. Selain itu, **prioritized experience replay** memberi prioritas pada pengalaman yang lebih jarang atau lebih bermanfaat, membantu mempercepat pembelajaran dengan fokus pada pengalaman yang lebih eksploratif. Terakhir, **meta-learning** dapat digunakan untuk mengajarkan agen cara mengelola eksplorasi dan eksploitasi secara efisien berdasarkan pengalaman sebelumnya. Semua pendekatan ini bertujuan untuk menyeimbangkan eksplorasi dan eksploitasi sehingga agen dapat memaksimalkan kinerjanya dan belajar kebijakan optimal dengan lebih cepat dan efektif.

- **Potensi aplikasi lain dari RL dalam sistem kendali nyata apa saja yang dapat diimplementasikan?**

Reinforcement Learning (RL) memiliki potensi aplikasi yang sangat luas dalam sistem kendali nyata, terutama dalam bidang-bidang yang melibatkan pengambilan keputusan sekuensial dalam lingkungan yang dinamis dan kompleks. Salah satu contoh aplikasi RL adalah dalam kendali robotik, di mana RL digunakan untuk mengendalikan robot industri, robot otonom, atau robot pembantu, memungkinkan agen untuk belajar mengoptimalkan tugas-tugas seperti manipulasi objek, navigasi, atau pengaturan gerakan tubuh. Selain itu, RL juga dapat diterapkan dalam kendaraan otonom, seperti mobil otonom dan drone, untuk mengoptimalkan rute perjalanan, menghindari tabrakan, dan beradaptasi dengan kondisi lalu lintas atau cuaca yang berubah. Dalam sistem pengaturan energi dan **smart grids**, RL dapat digunakan untuk mengelola distribusi energi secara efisien, mempertimbangkan faktor seperti permintaan energi dan ketersediaan sumber daya terbarukan. Dalam industri manufaktur, RL dapat digunakan untuk mengontrol berbagai proses produksi dengan memaksimalkan

efisiensi atau kualitas produk, sementara dalam sistem energi terbarukan, RL mengoptimalkan pengaturan panel surya atau turbin angin. Aplikasi RL juga sangat relevan dalam sistem kesehatan, seperti pengaturan ventilator untuk perawatan pasien atau penyesuaian dosis obat. Selain itu, RL dapat digunakan dalam pengendalian sistem penerbangan dan pelacakan satelit, pengelolaan jaringan telekomunikasi untuk mengoptimalkan penjadwalan dan alokasi sumber daya, serta dalam pengelolaan portofolio investasi dan trading sistem di pasar finansial. Dalam pengolahan data besar, RL membantu mengoptimalkan aliran dan pemrosesan data real-time. Aplikasi lain yang menjanjikan termasuk kendali di lingkungan berisiko tinggi seperti reaktor nuklir dan sistem pertahanan, di mana RL dapat membantu meminimalkan risiko dan meningkatkan keselamatan. Dengan berbagai aplikasi ini, RL berpotensi mengubah cara kita mengelola dan mengendalikan berbagai sistem di dunia nyata, memberikan solusi yang lebih efisien, adaptif, dan otonom.